University of British Columbia, Vancouver
Department of Computer Science

CPSC 304 Project Cover Page

Milestone #: 1

Date: February 4, 2026

Group Number: 13

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Spencer Liu	27641240	f9a3l	spencerliu07@gmail.com
Sean Yan	64322415	q2m3y	dhsyan@student.ubc.ca
William Lin	62611330	u7e0v	williamlin0640@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

Project Description

Domain

The app is focused on financial markets and investment analytics. It is a stock market tracking and portfolio management database that stores information about public companies and their stocks, the people involved, and how market performance changes over different time periods. The goal is to support typical “market tracker” questions that offer meaningful comparisons over a period of time.

What aspects of the domain are modeled by the database?

The database models the core entities and relationships needed to track securities, participants, portfolios, and time-based market performance, including:

Market instruments and issuers:
The schema models companies and their associated stocks This captures the structure needed for stock tracking.

People in the market ecosystem:
A People entity specializes (ISA) into roles like Traders and CEOs, allowing the system to store shared person data (name/email/net worth/cash) while also supporting role-specific attributes (trader performance/average trade time, other important user attributes).

Trading activity and ownership relationships:
The schema includes relationships for buying and selling (traders buying/selling stocks), and links between companies/people such as a CEO operating a company and a person/entity owning something with a percentage. This supports real-world use cases like reconstructing activity histories and representing equity/portfolio allocations.

Portfolios and holdings:
Users can track portfolios (with attributes like name, currency, created date) and how they contain stocks, reflecting a real investment workflow.

Time-based performance tracking:
The schema models a TimeFrame concept with ISA categories such as Days, Months, and Years, and attaches market metrics like average change and volume over those time windows.

Relational Schema

Underline = Primary Key
Bold = Foreign Key
[] or () in domain = Constraint

Entities:
Assets(TickerSymbol varchar(10) NOT NULL, Name varchar(100), LastPrice numeric(5,2), CurrentPrice numeric(5,2), Exchange varchar(20), LastUpdate varchar(20), IsActive int[0,1], Currency char(3), CreatedDate date)

TickerSymbol -> {Name, LastPrice, CurrentPrice, Exchange, LastUpdate, IsActive, Currency, CreatedDate}

Stocks(**TickerSymbol varchar(10) NOT NULL**, Industry varchar(100), Sector varchar(100), SharesOutstanding int[n≥0], EarningsPerShare numeric(5,2))

Portfolio(CreatedDate date, Name varchar(100), Currency char(3))

People(Email varchar(100) NOT NULL, Networth numeric(10,2), Name varchar(100), Cash numeric(10,2))

Traders(**Email varchar(100) NOT NULL**, Performance int[1-10], AverageTradeTime int)

CEOs(**Email varchar(100) NOT NULL**, InCompanyTime int, PublicConsensus int[0-100], PreviousRoles varchar(100))

Reporter(**Email varchar(100) NOT NULL**, PublicConsensus int[0-100])

Companies(CompanyID int NOT NULL, FoundedDate date, IsPublic int[0,1], EmployeeCount int[n≥0])

NewsEvents(EventID int NOT NULL, Headline varchar(100), SentimentScore[0,100], **TimeFrameID int**, Happened int)

TimeFrame(TimeFrameID int NOT NULL, Volume int, Time_to_time varchar(50), AverageChange int)

Days(**TimeFrameID int NOT NULL**, DayOfMonth int[1-31], DayOfWeek char(10), IsHoliday int[0,1], Has_Months int[1-12])

Months(**TimeFrameID int NOT NULL**, MonthOfYear int[1-12], Has_Years int)

Years(**TimeFrameID int NOT NULL**, YearNumber int)

Operations:

Contains(**Email varchar(100) NOT NULL**, **CreatedDate date**, **TickerSymbol varchar(10) NOT NULL,** Quantity int, PurchasePrice int)

Buys(**Email varchar(100) NOT NULL**, **TickerSymbol varchar(10) NOT NULL,** Quantity int)

Sells(**Email varchar(100) NOT NULL**, **TickerSymbol varchar(10) NOT NULL,** Quantity int)

Trades(**TickerSymbol varchar(10) NOT NULL**, **CompanyID int NOT NULL,** PublicDate date, PercentageOfCompany int[0,100])

People_Has_Portfolio(**CreatedDate date**, Email varchar(100))

Operates(**Email varchar(100) NOT NULL**, **CompanyID int NOT NULL**, IsFounder int[0,1], InOfficeTime int)

Owns(**Email varchar(100) NOT NULL**, **CompanyID int NOT NULL**, Percentage numeric(5,2))

Impacts(**TickerSymbol varchar(10) NOT NULL, EventID int NOT NULL,** PreEventPrice int, PostEventPrice int, **TimeFrameID int NOT NULL**)

Reports(**Email varchar(100) NOT NULL, EventID int NOT NULL)**

On(**TickerSymbol varchar(10) NOT NULL, EventID int NOT NULL, TimeFrameID int NOT NULL**)

Functional Dependencies

Key Entity Attributes:
TickerSymbol → Name, LastPrice, CurrentPrice, Exchange, LastUpdate, IsActive, Currency, CreatedDate, Industry, Sector, SharesOutstanding, EarningsPerShare

Email → Networth, Name, Cash, Performance, AverageTradeTime, InCompanyTime, PublicConsensus, PreviousRoles

CompanyID → FoundedDate, IsPublic, EmployeeCount

TimeFrameID → Volume, Time_to_time, AverageChange, DayOfMonth, DayOfWeek, IsHoliday, Has_Months, MonthOfYear, Has_Years, YearNumber

EventID → Headline, SentimentScore, TimeFrameID, Happened

Relation Attributes:
Email, CompanyID → Percentage, IsFounder, InOfficeTime

Email, CreatedDate, TickerSymbol → Contains.Quantity, PurchasePrice

Email, TickerSymbol → Buys.Quantity, Sells.Quantity

TickerSymbol, CompanyID, PublicDate → PercentageOfCompany

TickerSymbol, EventID, TimeFrameID → PreEventPrice, PostEventPrice

SQL DDL

```
CREATE TABLE Assets (
    TickerSymbol VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(100),
    LastPrice NUMERIC(10,2),
    CurrentPrice NUMERIC(10,2),
    Exchange VARCHAR(20),
    LastUpdate VARCHAR(20),
    IsActive INT CHECK (IsActive IN (0,1)),
    Currency CHAR(3),
    CreatedDate DATE
);

CREATE TABLE Stocks (
    TickerSymbol VARCHAR(10) PRIMARY KEY,
    Industry VARCHAR(100),
    Sector VARCHAR(100),
    SharesOutstanding INT CHECK (SharesOutstanding > 0),
    EarningsPerShare NUMERIC(10,2),
    FOREIGN KEY (TickerSymbol) REFERENCES Assets(TickerSymbol)
);

CREATE TABLE Portfolio (
    CreatedDate DATE PRIMARY KEY,
    Name VARCHAR(100),
    Currency CHAR(3)
);

CREATE TABLE People (
    Email VARCHAR(100) PRIMARY KEY,
    Networth NUMERIC(12,2),
    Name VARCHAR(100),
    Cash NUMERIC(12,2)
);

CREATE TABLE Traders (
    Email VARCHAR(100) PRIMARY KEY,
    Performance INT CHECK (Performance BETWEEN 1 AND 10),
    AverageTradeTime INT,
    FOREIGN KEY (Email) REFERENCES People(Email)
);
```

```
CREATE TABLE CEOs (
    Email VARCHAR(100) PRIMARY KEY,
    InCompanyTime INT,
    PublicConsensus INT CHECK (PublicConsensus BETWEEN 0 AND 100),
    PreviousRoles VARCHAR(100),
    FOREIGN KEY (Email) REFERENCES People(Email)
);

CREATE TABLE Reporter (
    Email VARCHAR(100) PRIMARY KEY,
    PublicConsensus INT CHECK (PublicConsensus BETWEEN 0 AND 100),
    FOREIGN KEY (Email) REFERENCES People(Email)
);

CREATE TABLE Companies (
    CompanyID INT PRIMARY KEY,
    FoundedDate DATE,
    IsPublic INT CHECK (IsPublic IN (0,1)),
    EmployeeCount INT CHECK (EmployeeCount > 0)
);

CREATE TABLE TimeFrame (
    TimeFrameID INT PRIMARY KEY,
    Volume INT,
    Time_to_time VARCHAR(50),
    AverageChange INT
);

CREATE TABLE Days (
    TimeFrameID INT PRIMARY KEY,
    DayOfMonth INT CHECK (DayOfMonth BETWEEN 1 AND 31),
    DayOfWeek VARCHAR(10),
    IsHoliday INT CHECK (IsHoliday IN (0,1)),
    Has_Months INT CHECK (Has_Months BETWEEN 1 AND 12),
    FOREIGN KEY (TimeFrameID) REFERENCES TimeFrame(TimeFrameID)
);

CREATE TABLE Months (
    TimeFrameID INT PRIMARY KEY,
    MonthOfYear INT CHECK (MonthOfYear BETWEEN 1 AND 12),
    Has_Years INT,
    FOREIGN KEY (TimeFrameID) REFERENCES TimeFrame(TimeFrameID)
);
```

```
CREATE TABLE Years (
    TimeFrameID INT PRIMARY KEY,
    YearNumber INT,
    FOREIGN KEY (TimeFrameID) REFERENCES TimeFrame(TimeFrameID)
);

CREATE TABLE NewsEvents (
    EventID INT PRIMARY KEY,
    Headline VARCHAR(100),
    SentimentScore INT CHECK (SentimentScore BETWEEN 0 AND 100),
    TimeFrameID INT,
    Happened INT CHECK (Happened IN (0,1)),
    FOREIGN KEY (TimeFrameID) REFERENCES TimeFrame(TimeFrameID)
);

CREATE TABLE Contains (
    Email VARCHAR(100),
    CreatedDate DATE,
    TickerSymbol VARCHAR(10),
    Quantity INT,
    PurchasePrice INT,
    PRIMARY KEY (Email, CreatedDate, TickerSymbol),
    FOREIGN KEY (Email) REFERENCES People(Email),
    FOREIGN KEY (CreatedDate) REFERENCES Portfolio(CreatedDate),
    FOREIGN KEY (TickerSymbol) REFERENCES Assets(TickerSymbol)
);

CREATE TABLE Buys (
    Email VARCHAR(100),
    TickerSymbol VARCHAR(10),
    Quantity INT,
    PRIMARY KEY (Email, TickerSymbol),
    FOREIGN KEY (Email) REFERENCES People(Email),
    FOREIGN KEY (TickerSymbol) REFERENCES Assets(TickerSymbol)
);

CREATE TABLE Sells (
    Email VARCHAR(100),
    TickerSymbol VARCHAR(10),
    Quantity INT,
    PRIMARY KEY (Email, TickerSymbol),
    FOREIGN KEY (Email) REFERENCES People(Email),
    FOREIGN KEY (TickerSymbol) REFERENCES Assets(TickerSymbol)
);
```

```
CREATE TABLE Trades (
    TickerSymbol VARCHAR(10),
    CompanyID INT,
    PublicDate DATE,
    PercentageOfCompany INT CHECK (PercentageOfCompany BETWEEN 0 AND 100),
    PRIMARY KEY (TickerSymbol, CompanyID, PublicDate),
    FOREIGN KEY (TickerSymbol) REFERENCES Assets(TickerSymbol),
    FOREIGN KEY (CompanyID) REFERENCES Companies(CompanyID)
);

CREATE TABLE People_Has_Portfolio (
    CreatedDate DATE,
    Email VARCHAR(100),
    PRIMARY KEY (CreatedDate, Email),
    FOREIGN KEY (CreatedDate) REFERENCES Portfolio(CreatedDate),
    FOREIGN KEY (Email) REFERENCES People(Email)
);

CREATE TABLE Operates (
    Email VARCHAR(100),
    CompanyID INT,
    IsFounder INT CHECK (IsFounder IN (0,1)),
    InOfficeTime INT,
    PRIMARY KEY (Email, CompanyID),
    FOREIGN KEY (Email) REFERENCES CEOs(Email),
    FOREIGN KEY (CompanyID) REFERENCES Companies(CompanyID)
);

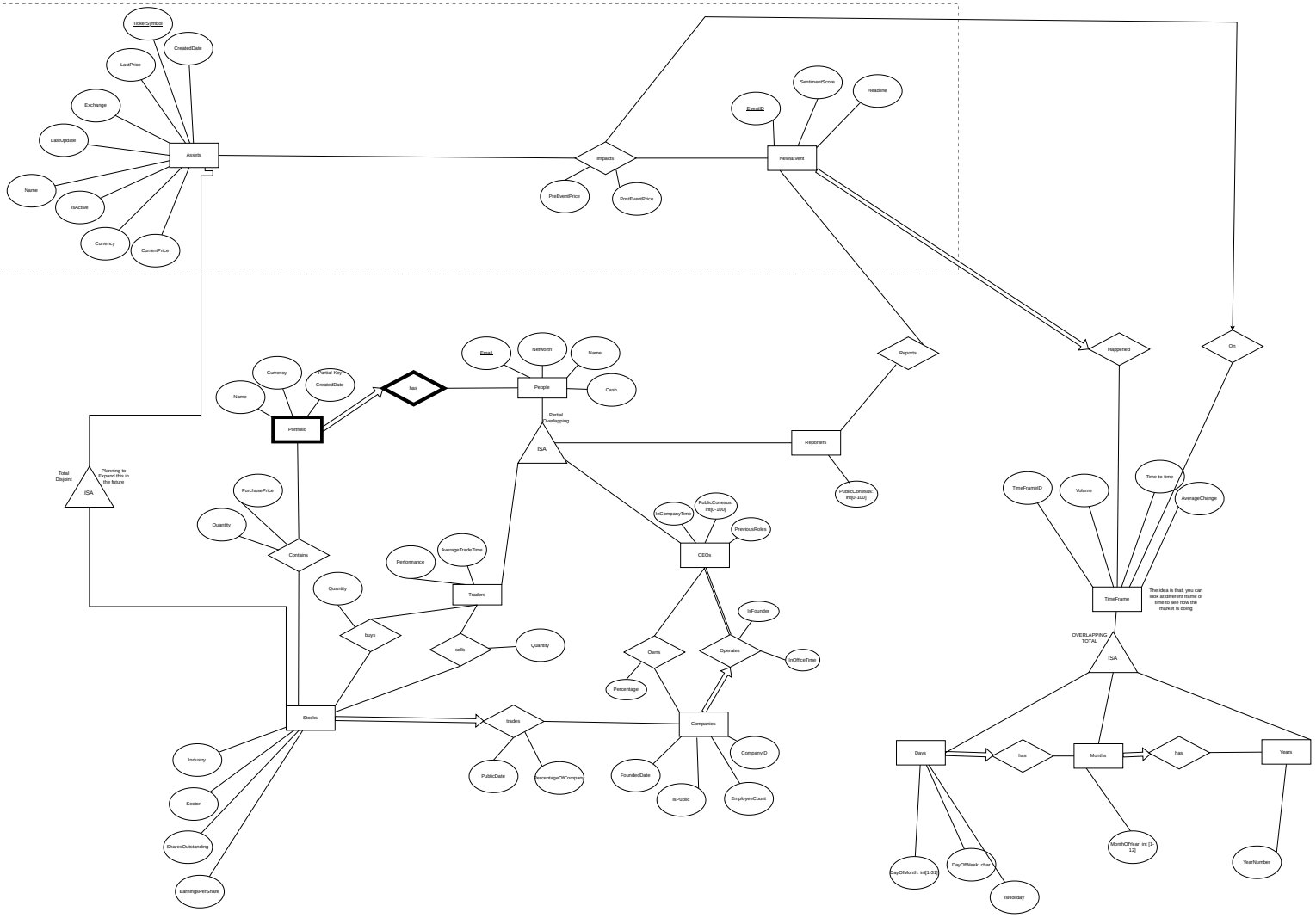
CREATE TABLE Owns (
    Email VARCHAR(100),
    CompanyID INT,
    Percentage NUMERIC(5,2),
    PRIMARY KEY (Email, CompanyID),
    FOREIGN KEY (Email) REFERENCES People(Email),
    FOREIGN KEY (CompanyID) REFERENCES Companies(CompanyID)
);
```

```
CREATE TABLE Impacts (
    TickerSymbol VARCHAR(10),
    EventID INT,
    PreEventPrice INT,
    PostEventPrice INT,
    PRIMARY KEY (TickerSymbol, EventID),
    FOREIGN KEY (TickerSymbol) REFERENCES Assets(TickerSymbol),
    FOREIGN KEY (EventID) REFERENCES NewsEvents(EventID)
);
CREATE TABLE OnRelation (
    TickerSymbol VARCHAR(10),
    EventID INT,
    PRIMARY KEY (TickerSymbol, EventID),
    FOREIGN KEY (TickerSymbol) REFERENCES Assets(TickerSymbol),
    FOREIGN KEY (EventID) REFERENCES NewsEvents(EventID)
);
```

Declarations

We will use OracleDB for the database, with Javascript for the language.

No AI was used in this milestone.


TickerSymbol
CreatedDate
LastPrice
Exchange
LastUpdate
Name
IsActive
Currency
CurrentPrice
Assets
Impacts
PreEventPrice
PostEventPrice
NewsEvent
EventID
SentimentScore
Headline
Happened
On
Reports
Email
Networth
Name
Cash
People
has
Currency
Partial-Key
CreatedDate
Name
Portfolio
Partial Overlapping
ISA
Reporters
PublicConsesus: int[0-100]
Total Disjoint
ISA
Planning to Expand this in the future
PurchasePrice
Quantity
Contains
Quantity
buys
sells
Quantity
Performance
AverageTradeTime
Traders
inCompanyTime
PublicConsesus: int[0-100]
PreviousRoles
CEOs
IsFounder
Owns
Operates
inOfficeTime
Percentage
Companies
trades
PublicDate
PercentageOfCompany
FoundedDate
IsPublic
EmployeeCount
CompanyID
Stocks
Industry
Sector
SharesOutstanding
EarningsPerShare
TimeFrameID
Volume
Time-to-time
AverageChange
TimeFrame
The idea is that, you can look at different frame of time to see how the market is doing
OVERLAPPING TOTAL
ISA
Days
has
Months
has
Years
DayOfMonth: int[1-31]
DayOfWeek: char
IsHoliday
MonthOfYear: int [1-12]
YearNumber